

Task Manager API

Este projeto consiste numa API REST para gestão de tarefas, desenvolvida em Node.js com Express e MySQL. A aplicação permite autenticação de utilizadores através de JWT (JSON Web Token) e operações CRUD sobre tarefas, garantindo que cada utilizador apenas pode aceder às suas próprias informações. Para o desenvolvimento da API foi utilizada uma abordagem Code First, onde a implementação do backend e das rotas foi realizada inicialmente em código. Posteriormente, a documentação Swagger/OpenAPI foi gerada automaticamente através de anotações @openapi inseridas nos controllers da aplicação.

Tecnologias Utilizadas

- Node.js
- Express.js
- MySQL
- Docker
- Docker Compose
- JWT (JSON Web Token)
- Swagger/OpenAPI
- mysql2
- dotenv

Estrutura do Projeto

```
task-manager-api/  
db/  
  Dockerfile.db  
  init.sql  
src/  
  config/  
    db.js  
  controllers/  
    authController.js  
    taskController.js  
    categoryController.js  
  middlewares/  
    auth.js  
  routes/  
    index.js  
Dockerfile  
docker-compose.yml  
package.json  
server.js  
README.md
```

- `config/` -> configuração da ligação MySQL
- `controllers/` -> lógica das rotas e queries SQL
- `middlewares/` -> middleware de autenticação JWT
- `routes/` -> definição dos endpoints da API
- `db/` -> scripts e configuração da base de dados

Base de Dados

A aplicação utiliza MySQL como sistema de gestão de base de dados. A base de dados é criada automaticamente através do ficheiro `init.sql`, executado pelo container Docker do MySQL durante a inicialização da aplicação.

Recursos Implementados

Foram implementados 3 recursos principais:

- Utilizadores (`users`)
- Tarefas (`tasks`)
- Categorias (`categories`)

Além disso, foi criada uma tabela intermédia:

- `task_categories`

para representar a relação entre tarefas e categorias.

Relações entre Recursos

Relação 1:N

Existe uma relação de cardinalidade **1:N** entre:

- `users`
- `tasks`

Um utilizador pode possuir várias tarefas, enquanto cada tarefa pertence apenas a um utilizador.

Essa relação é implementada através da chave estrangeira:

```
user_id INT,  
FOREIGN KEY (user_id) REFERENCES users(id)
```

Autenticação e Autorização

A autenticação da API foi implementada utilizando JWT (JSON Web Token). O utilizador realiza login através `/login`, fornecendo email e password. Após validação das credenciais na base de dados, o servidor gera um token JWT assinado com uma chave secreta (`JWT_SECRET`). Esse token é posteriormente enviado pelo cliente no header HTTP Authorization utilizando o formato:

```
Authorization: Bearer <token>
```

O middleware de autenticação verifica:

- existência do token
- validade da assinatura
- expiração do token

Após validação, os dados do utilizador são inseridos em `req.user`, permitindo controlar o acesso aos recursos.

A autorização foi implementada utilizando o `user_id` associado às tarefas. Dessa forma:

- um utilizador apenas consegue visualizar as suas tarefas
- um utilizador apenas consegue editar as suas tarefas
- um utilizador apenas consegue apagar as suas tarefas

Isto é garantido através de queries SQL que filtram pelo `user_id` presente no token JWT.

Comparação com OAuth2

A autenticação implementada neste projeto utiliza JWT simples com autenticação local baseada em email e password.

Diferentemente do OAuth2, esta solução não utiliza:

- Authorization Server
- Refresh Tokens
- Scopes
- consentimento de terceiros
- login federado (Google, Facebook, GitHub)

No OAuth2, existem diferentes authorization flows, como:

- Authorization Code Flow
- Client Credentials Flow
- Implicit Flow
- Device Flow

Esses flows são utilizados principalmente quando aplicações precisam de autenticação delegada entre diferentes serviços ou plataformas externas. Neste projeto foi utilizada uma abordagem mais simples baseada em JWT, adequada para APIs REST pequenas e médias, onde a própria aplicação é responsável pela autenticação dos utilizadores.

Docker

A aplicação utiliza uma arquitetura multi-container:

- container Node.js
- container MySQL

Execução com Docker

Para iniciar os containers:

```
docker-compose up --build
```

A API ficará disponível em:

```
http://localhost:3000
```

Swagger:

```
http://localhost:3000/api-docs
```

Swagger JSON

```
http://localhost:3000/swagger.json
```

Endpoints

Endpoints

Método	Endpoint	Descrição
POST	/login	Login
GET	/tasks	Listar tarefas
POST	/tasks	Criar tarefa
PUT	/tasks/:id	Atualizar tarefa
DELETE	/tasks/:id	Apagar tarefa

Método	Endpoint	Descrição
GET	/categories	Listar categorias

Segurança

A aplicação implementa:

- autenticação JWT
- middleware de autorização
- proteção de rotas privadas
- isolamento de tarefas por utilizador
- utilização de variáveis de ambiente com dotenv
- verificação de token Bearer

Além disso, o detalhe do utilizador autenticado é apresentado na consola sempre que um pedido autenticado é recebido:

```
Acesso autorizado: Igor Silva (ID: 1)
```

Documentação da API

A documentação da API foi implementada utilizando Swagger/OpenAPI 3.0.

As anotações `@openapi` foram inseridas diretamente nos controllers da aplicação, permitindo gerar automaticamente:

- documentação Swagger UI
- ficheiro `swagger.json`

Collection Postman

O projeto inclui uma collection do Postman para facilitar os testes da API.

A collection permite testar:

- login
- autenticação JWT
- CRUD de tarefas
- consulta de categorias

A collection pode ser importada diretamente no Postman.

Conclusão

Este projeto permitiu desenvolver uma API REST completa utilizando Node.js, Express e MySQL, aplicando conceitos de autenticação, autorização e documentação de APIs. Foram implementadas operações CRUD protegidas através de JWT, garantindo que cada utilizador apenas consegue aceder aos seus próprios recursos.